

Section 10.3 - Understanding RNN layers in Keras

December 13, 2025

1 Understanding recurrent neural networks

- Iterate through the sequence elements and maintaining a state that contains information relative to what it has seen so far
- Build from past information and constant updates as new information comes in
- RNN is characterized by the step function
- RNN has an internal loop over sequence elements
- Data point is no longer processed in a single step
- The state is reset for every new sample

1.1 NumPy implementation of a toy RNN

- Input - a rank-2 tensor (timesteps, features)
- Each timestep t: current state at t & input at t yield output at t
- NumPy arrays iterate over its first axis by default
- tanh activation - centered at zero and bounded range in (-1,1) (unlike ReLU)
- state_0 = 0

```
[1]: import numpy as np

timesteps = 100
input_features = 32
output_features = 64
inputs = np.random.random((timesteps, input_features))
state_t = np.zeros((output_features,))
W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))
successive_outputs = []
for input_t in inputs:
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
    successive_outputs.append(output_t)
    state_t = output_t
final_output_sequence = np.stack(successive_outputs, axis=0)
```

1.2 A recurrent layer in Keras

- Understand the SimpleRNN layer - process batches of sequences

- Recurrent layers in Keras: SimpleRNN, Long Short-Time Memory (LSTM) and Gated Recurrent Unit (GRU)
- Output: the entire sequence or the last output only

1.2.1 Last output step only

```
[2]: from tensorflow import keras
from tensorflow.keras import layers

num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16, return_sequences=False)(inputs)
print(outputs.shape)
```

(None, 16)

1.2.2 Full output sequence

```
[3]: num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16, return_sequences=True)(inputs)
print(outputs.shape)
```

(None, 120, 16)

1.2.3 Stacking RNN layers

```
[4]: inputs = keras.Input(shape=(steps, num_features))
x = layers.SimpleRNN(16, return_sequences=True)(inputs)
x = layers.SimpleRNN(16, return_sequences=True)(x)
outputs = layers.SimpleRNN(16)(x)
```

1.3 The SimpleRNN layer

$$h_t = \tanh(W * input_t + U * h_{t-1} + b)$$

- $input_t$ - input features at timestep t
- h_t - layer output (also, hidden state!) at timestep t
- h_{t-1} - layer output at timestep $t-1$ (also, state _{t}) - Short-Term Memory
- W , U , b are tensor weights
- Vanishing gradient problem - not used in practice.

1.4 The Long Short-Term Memory (LSTM) layer

$$h_t = output_gate_t * \tanh(c_t)$$

- 1) Carry update (Long-Term Memory): $c_t = f_t * c_{t-1} + i_t * k_t$

- Forget gate: $f_t = \text{activation}(W_f * input_t + U_f * h_{t-1} + b_f)$

- Input gate: $i_t = \text{activation}(W_i * \text{input}_t + U_i * h_{t-1} + b_i)$
 - Candidate memory: $k_t = \tanh(W_k * \text{input}_t + U_k * h_{t-1} + b_k)$
 - input_t - input features at timestep t
 - h_t - layer output (also, hidden state!) at timestep t
 - h_{t-1} - layer output at timestep $t-1$ (also, state_t) - Short-Term Memory
 - W_* , U_* , b_* are tensor weights
- 2) Output gate: $\text{output_gate}_t = \text{activation}(W_o * \text{input}_t + U_o * h_{t-1} + b_o)$
- 3) Initial states: $h_0 = 0$, $c_0 = 0$

1.5 The Gated Recurrent Unit (GRU) layer

$$h_t = (1 - z_t) * h_{t-1} + z_t * k_t$$

- 1) Update gate: $z_t = \text{activation}(W_z * \text{input}_t + U_z * h_{t-1} + b_z)$
- z_t small - long-term memory
 - z_t large - short-term update
 - input_t - input features at timestep t
 - h_t - layer output (also, hidden state!) at timestep t
 - h_{t-1} - layer output at timestep $t-1$ (also, state_t) - Short-Term Memory
 - W_* , U_* , b_* are tensor weights
- 2) Reset gate: $r_t = \text{activation}(W_r * \text{input}_t + U_r * h_{t-1} + b_r)$
- 3) Candidate activation: $k_t = \tanh(W_k * \text{input}_t + U_k * (r_t * h_{t-1}) + b_k)$
- 4) Initial states: $h_0 = 0$